Print copy of official Microsoft Learn documentation

Source: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_modules?view=powershell-7.6
Exported: 2026-10-14. Source metadata date: 2024-10-09T00:00:00Z.

Microsoft and documentation contributors. Article wording and code for the requested version are retained; web navigation is removed and print formatting is added. This locally generated PDF is not a Microsoft-published PDF book or a complete product manual. Source-page attribution and notices apply.

Documentation license and terms: Microsoft Learn terms; .NET documentation: CC BY 4.0 license. Check the source for specific attribution or exceptions.

about_Modules

Short description

Explains how to install, import, and use PowerShell modules.

Long description

PowerShell is both a command shell and a scripting language. Commands in PowerShell are implemented as scripts, functions, or cmdlets. The language includes keywords, which provide the structure and logic of processing, and other resources, such as variables, providers, aliases.

A module is a self-contained, reusable unit that can include cmdlets, providers, functions, variables, and other resources. By default, PowerShell automatically loads an installed module the first time you use a command from the module. You can configure automatic module loading behavior using the variable `$PSModuleAutoLoadingPreference`. For more information, see about_Preference_Variables.

You can also manually load or unload modules during a PowerShell session. To load or reload a module, use `Import-Module`. To unload a module, use the `Remove-Module` cmdlet.

PowerShell includes a base set of modules. Anyone can create new modules using C# or the PowerShell scripting language itself. Modules written in C# as compiled .NET assemblies are known as native modules. Modules written in PowerShell are known as script modules.

This article explains how to use PowerShell modules. For information about how to create PowerShell modules, see Writing a PowerShell Module.

> Note

Prior to PowerShell 3.0, cmdlets and providers were packaged in PowerShell snap-ins. Beginning in PowerShell 3.0, the **Microsoft.PowerShell.Core** snap-in is added to every session by default. This is the only snap-in remaining in PowerShell. All other snap-ins were converted to modules. Creation of new snap-ins is no longer supported.

Default module locations

PowerShell stores modules in the following default locations:

- On Windows
  - All users scope - `$Env:ProgramFiles\PowerShell\Modules`
  - Current user scope - `$HOME\Documents\PowerShell\Modules`
  - Modules shipped with PowerShell - `$PSHOME\Modules`
- On Linux and macOS
  - All users scope - `/usr/local/share/powershell/Modules`
  - Current user scope - `$HOME/.local/share/powershell/Modules`
  - Modules shipped with PowerShell - `$PSHOME/Modules`

By default, the `Modules` folder for the current user doesn't exist. If you installed a module in the `CurrentUser` scope using `Install-Module` or `Install-PSResource`, those cmdlets create the `Modules` folder for the current user. If the folder doesn't exist, you can create it manually.

Use the following command to create a `Modules` folder for the current user:

```
$folder = New-Item -Type Directory -Path $HOME\Documents\PowerShell\Modules
```

These locations are automatically included in the `$Env:PSModulePath` environment variable. For more information about the default module locations, see about_PSModulePath.

Module autoloading

The first time that you run a command from an installed module, PowerShell automatically imports (loads) that module. The module must be stored in the locations specified in the `$Env:PSModulePath` environment variable.

Module autoloading allows you to use commands in a module without any setup or profile configuration. Each of the following examples causes the **CimCmdlets** module, which contains `Get-CimInstance`, to be imported into your session.

- Run the Command

  ```
  Get-CimInstance Win32_OperatingSystem
  ```

- Get the Command

  ```
  Get-Command Get-CimInstance
  ```

- Get Help for the Command

```
Get-Help Get-CimInstance
```

When you use `Get-Command` with a wildcard character (*), PowerShell doesn't import any modules. You can use wildcards for command discovery without loading modules that you might not need in your session.

Manually import a module

Manually importing a module is required when a module isn't installed in the locations specified by the `$Env:PSModulePath` environment variable, or when the module is provided as a standalone `.dll` or `.psm1` file, rather than a packaged module.

Also, commands that use PowerShell providers don't automatically import a module. For example, if you use a command that requires the `WSMan:` drive, such as the `Get-PSSessionConfiguration` cmdlet, you might need to run the `Import-Module` cmdlet to import the **Microsoft.WSMan.Management** module that includes the `WSMan:` drive.

You might also want to change how the module is imported in your session. For example, the **Prefix** parameter of `Import-Module` adds a distinctive prefix to the noun portion of the cmdlets imported from the module. The **NoClobber** parameter prevents the module from adding commands that would hide or replace existing commands in the session. For more information, see Manage name conflicts.

The following example imports the **BitsTransfer** module into the current session.

```
Import-Module BitsTransfer
```

To import a module that isn't in your `$Env:PSModulePath`, use the fully qualified path to the module folder. For example, to add the **TestCmdlets** module in the `C:\ps-test` directory to your session, type:

```
Import-Module C:\ps-test\TestCmdlets
```

To import a module file that isn't contained in a module folder, use the fully qualified path to the module file in the command. For example, to add the TestCmdlets.dll module in the `C:\ps-test` directory to your session, type:

```
Import-Module C:\ps-test\TestCmdlets.dll
```

For more information about adding modules to your session, see Import-Module.

Import a module at the start of every session

The `Import-Module` command imports modules into your current PowerShell session. To import a module into every PowerShell session that you start, add the `Import-Module` command to your PowerShell profile.

For more information about profiles, see about_Profiles.

Install a published module

A published module is a module that's available from a registered repository, such as the PowerShell Gallery. The **PowerShellGet** and **Microsoft.PowerShell.PSResourceGet** modules provide cmdlets for finding, installing, and publishing PowerShell modules to a registered repository.

The **PowerShellGet** module is included with PowerShell 5.0 and later releases. The **Microsoft.PowerShell.PSResourceGet** module is included with PowerShell 7.4 and later releases and is the preferred package manager for PowerShell. **Microsoft.PowerShell.PSResourceGet** can be installed, side by side with **PowerShellGet**, on older versions of PowerShell. Use the `Install-Module` or `Install-PSResource` cmdlet to install modules from the PowerShell Gallery.

```
 Get-Command Install-Module, Install-PSResource
```

```
CommandType  Name               Version    Source
-----------  ----               -------    ------
Function     Install-Module     2.9.0      PowerShellGet
Cmdlet       Install-PSResource 1.0.0      Microsoft.PowerShell.PSResourceGet
```

For more information, see PowerShellGet Overview.

Manually install a module

You can manually install a module by copying the module contents from another folder. That folder can be in another location on the local machine or installed on another machine. To install a module manually, copy the entire module folder into a new location included in your `$Env:PSModulePath`.

In PowerShell use the `Copy-Item` cmdlet. For example, run the following command to copy the `MyModule` folder from `C:\PSTest`:

```
$modulePath = $HOME\Documents\PowerShell\Modules\MyModule
Copy-Item -Path C:\PSTest\MyModule\* -Destination $modulePath -Recurse
```

You can install a module in any location, but installing your modules in a default module location makes them easier to manage.

Find installed modules

The `Get-Module` cmdlet gets the PowerShell modules that are loaded in the current PowerShell session.

```
Get-Module
```

The modules listed can include modules that were imported from any location, not just from `$Env:PSModulePath`.

Use the following command to list modules that are installed in the `$Env:PSModulePath`:

```
Get-Module -ListAvailable
```

This command gets all modules that are installed in `$Env:PSModulePath`, not just the modules that are imported into the current session. This command doesn't list modules that are installed in other locations.

For more information, see Get-Module.

List the commands in a module

Use the `Get-Command` cmdlet to find all available commands. You can use the parameters of the `Get-Command` cmdlet to filter commands such as by module, name, and noun.

To find all commands in a module, type:

```
Get-Command -Module <module-name>
```

For example, to find the commands in the **BitsTransfer** module, type:

```
Get-Command -Module BitsTransfer
```

For more information about the `Get-Command` cmdlet, see Get-Command.

Remove a module

When you remove a module, the commands that the module added are deleted from the session. For example, the following command removes the **BitsTransfer** module from the current session.

```
Remove-Module BitsTransfer
```

Removing a module reverses the operation of importing a module. Removing a module doesn't uninstall the module. For more information, see Remove-Module.

Commands can be added to your session from modules and snap-ins. Modules can add all types of commands, including cmdlets, providers, and functions, and items, such as variables, aliases, and PowerShell drives. Snap-ins can add only cmdlets and providers.

Before removing a module from your session, use the following commands to determine which module you want to remove.

For example, use the following command to find the source of the `Get-Date` and `Get-Help` cmdlets:

```
Get-Command Get-Date, Get-Help -All |
    Select-Object -Property Name, CommandType, Module ,PSSnapIn
```

The following output shows that the `Get-Help` cmdlet is in the **Microsoft.PowerShell.Core** snap-in. This snap-in can't be removed from the session.

```
Name     CommandType Module                      PSSnapIn
----     ----------- ------                      --------
Get-Date    Function
Get-Date      Cmdlet Microsoft.PowerShell.Utility
Get-Help      Cmdlet                            Microsoft.PowerShell.Core
```

There are two sources for `Get-Date`. One is a function and the other is a cmdlet in the **Microsoft.PowerShell.Utility** module. You can remove the module using `Remove-Module`. To remove the function, you can delete it from the `Function:` drive.

```
Remove-Item Function:Get-Date
```

For more information about the `Function:` drive, see about_Function_Provider.

Manage name conflicts

Name conflicts occur when more than one command in the session has the same name. Importing a module causes a name conflict when commands in the module have the same names as commands or items in the session.

`Import-Module` might add commands that hide and replace commands in the current session. Name conflicts can result in commands being hidden or replaced. Command replacement occurs when the imported module contains a command with the same name as an existing command in the session. The newly imported command takes precedence over the existing command.

For example, when a session includes a function and a cmdlet with the same name, PowerShell runs the function by default. When the session includes commands of the same type with the same name, such as two cmdlets with the same name, by default, it runs the most recently added command.

For more information, including an explanation of the precedence rules and instructions for running hidden commands, see about_Command_Precedence.

You can run a hidden or replaced command by qualifying the command name. To qualify the command name, add the name of module that contains the version of the command you want. For example:

```
Microsoft.PowerShell.Utility\Get-Date
```

Running `Get-Date` with the module name prefix ensures that are running the version from the **Microsoft.PowerShell.Utility** module.

To detect name conflicts, use the **All** parameter of the `Get-Command` cmdlet. By default, `Get-Command` gets only that commands that run when you type the command name. The **All** parameter gets all commands with the specific name in the session.

To prevent name conflicts, use the **NoClobber** or **Prefix** parameters of the `Import-Module` cmdlet. The **Prefix** parameter adds a prefix to the names of imported commands so that they're unique in the session. The **NoClobber** parameter doesn't import any commands that would hide or replace existing commands in the session.

You can also use the **Alias**, **Cmdlet**, **Function**, and **Variable** parameters of `Import-Module` to select only the commands that you want to import, and you can exclude commands that cause name conflicts in your session.

Module authors can prevent name conflicts by using the **DefaultCommandPrefix** property of the module manifest to add a default prefix to all command names. The value of the **Prefix** parameter takes precedence over the value of **DefaultCommandPrefix**.

See also

- about_Command_Precedence
- about_Group_Policy_Settings
- Get-Command
- Get-Help
- Get-Module
- Import-Module
- Remove-Module
- Install-Module
- Install-PSResource